پروژه درس برنامه‌نویسی اندروید – ارائه سر کلاس

کلاس Event

مدل داده اپلیکیشن رویداد دانشگاهی – توضیح کامل و ساده، از صفر و با زبان ساده

com.mehrsamweb.universityevents | Event.java

```
public class Event {
    private String title;
    private String date;
    ...
}
```

Data Model | Android | Java

// 01

این فایل اصلاً چیست؟

یک کلاس ساده جاوا به نام `Event` است — خیلی ساده‌تر از `MainActivity`، ولی از نظر مفهومی پایه و اساس کل برنامه.

۱

کلاس چیست؟

کلاس مثل یک قالب یا فرم چاپی است؛ فرم خودش اطلاعات ندارد، فقط جاهای خالی دارد. هر بار که فرم را پر کنی، یک نسخه (شیء) از آن داری.

- کلاس `Event` = یک فرم خالی برای ثبت رویداد
- هر شیء `Event` = یک فرم پرشده (مثلاً کارگاه برنامه‌نویسی)

۲

`Event` یعنی رویداد

یک رویداد دانشگاهی مثل کارگاه برنامه‌نویسی، مسابقه طراحی وب، همایش فرهنگی.

- کارگاه برنامه‌نویسی
- مسابقه طراحی وب
- همایش فرهنگی

۳

نقش این کلاس

این کلاس مدل رویداد است — به این نوع کلاس‌ها می‌گویند `Model` یا مدل داده (Data Model).

چیزی روی صفحه نمایش نمی‌دهد؛ کارش فقط نگهداری اطلاعات است، مثل یک جعبه که اطلاعات یک رویداد داخلش می‌ریزیم.

// 02

package چیست؟

Event.java

```
package com.mehrsamweb.universityevents;
```

com › mehrsamweb › universityevents › Event.java

یعنی چه؟

این خط می‌گوید این فایل در کدام پوشه قرار دارد.

universityevents/
Event.java

تشبیه

مثل آدرس خانه است؛ مثل این است که بالای دفترت بنویسی «کلاس ۱۰۳، دبیرستان شهید بهشتی».

بالای دفتر: «کلاس ۱۰۳، دبیرستان شهید بهشتی»

نکته مهم

همین پکیجی است که MainActivity هم در آن بود — یعنی همه فایل‌های برنامه در یک پوشه هستند و می‌توانند همدیگر را ببینند.

Event.java MainActivity

همه فایل‌های برنامه در یک پوشه

// 03

شروع تعریف کلاس — public class Event

Event.java

```
public class Event {
```

> «من یک کلاس به نام Event می‌سازم که برای همه قابل استفاده است (public).»

public یعنی چه؟

یعنی این کلاس از بیرون هم قابل استفاده است؛ MainActivity می‌تواند آن را ببیند و از آن شیء بسازد. اگر public نمی‌گذاشتیم، فقط فایل‌های داخل همان پوشه می‌توانستند استفاده کنند. برای این برنامه فرقی نمی‌کرد، ولی public بودن عادت خوبی است.

نکته مهم

برخلاف MainActivity که وارث AppCompatActivity بود، این کلاس وارث هیچ کلاسی نیست — هیچ extends ندارد؛ یک کلاس خالص و مستقل است.

خالص و مستقل — بدون هیچ extends

Event → بدون extends | MainActivity → extends AppCompatActivity

متغیرهای داخل کلاس – فیلدها (ویژگی‌ها)

این پنج خط، ویژگی‌های یک رویداد هستند؛ یعنی هر رویداد باید این پنج چیز را داشته باشد.

Event.java

```
private String title;
private String date;
private String location;
private String category;
private String description;
```

۵ فیلد ✓ | همگی private ✓ | همگی String ✓

ویژگی	معنی	مثال
title	عنوان رویداد	«کارگاه برنامه‌نویسی اندروید»
date	تاریخ برگزاری	«1405/08/25»
location	محل برگزاری	«کارگاه کامپیوتر»
category	دسته‌بندی	«آموزشی»
description	توضیحات	«آشنایی با ساخت اپلیکیشن‌های اندرویدی»

مثال‌ها همان رویداد نمونه‌ی برنامه هستند.

هر ردیف جدول، دقیقاً همان یک خط از کد است.

private یعنی چه؟ — کپسوله‌سازی (Encapsulation)

مفهوم private

یعنی فقط داخل همین کلاس می‌شود به آن دسترسی داشت؛ از بیرون کسی نمی‌تواند مستقیم آن را ببیند یا تغییر دهد.

تشبیه: مثل یک کشو قفل‌دار در میز کارت — فقط خودت کلید داری؛ هر کس بخواهد محتوایش را ببیند یا عوض کند باید از تو اجازه بگیرد (از طریق Getter و Setter).

String چیست؟

نوع داده متن است؛ این ویژگی‌ها متن نگه می‌دارند — نه عدد، نه تاریخ واقعی. تاریخ را فعلاً به صورت متن ذخیره می‌کنیم و در جلسات بعد می‌توانیم به نوع داده دقیق‌تری تبدیلش کنیم.

```
private String title;
```

چرا کپسوله‌سازی خوب است؟

۱ امنیت

کسی نمی‌تواند مثلاً title را بی‌اجازه به null تبدیل کند.

۲ کنترل

اگر بعداً قانونی بگذاریم (مثل «عنوان نمی‌تواند خالی باشد»)، فقط یک جا یعنی در Setter اعمال می‌شود.

۳ انعطاف

اگر نحوه ذخیره را عوض کنیم، کاربران بیرونی متوجه نمی‌شوند.

Event.java

سازنده (Constructor) — مهم‌ترین بخش کلاس

Event.java

```java
public Event(String title, String date, String location, String category, String description) {
    this.title = title;
    this.date = date;
    this.location = location;
    this.category = category;
    this.description = description;
}
```

> وقتی یک شیء `Event` می‌سازیم، این ۵ خط اطلاعات داده‌شده را داخل ویژگی‌های شیء ذخیره می‌کنند؛ یعنی شیء از این به بعد این اطلاعات را به یاد دارد.

سازنده چیست؟

یک متد ویژه است که دقیقاً وقتی شیء جدید می‌سازیم (`new Event(...)`) اجرا می‌شود. تشبیه: مثل فرم پرکنی است — «فرم را با این اطلاعات پر کن».

چطور تشخیص دهیم؟

اسمش دقیقاً مثل اسم کلاس است (`Event`) و هیچ نوع بازگشتی ندارد (نه `void`، نه `String`). **این نکته طلایی:** اگر کسی پرسید از کجا بفهمیم، همین دو نشانه را بگو.

چرا ۵ پارامتر؟

چون ۵ ویژگی داریم؛ برای ساختن رویداد باید هر ۵ مقدار را بدهیم — همان کدی که در `MainActivity` دیدیم.

Event.java

this.title = title — دقیقاً چه کار می‌کند؟

Event.java

```
this.title = title;
```

سمت چپ — this.title

یعنی ویژگی title همین شیء (همین رویداد) — «عنوانِ من».

علامت =

یعنی مقدار سمت راست را بریز داخل سمت چپ.

سمت راست — title

یعنی پارامتری که از بیرون به سازنده داده شده.

تشبیه

جعبه‌ای داری با برچسب «عنوان» (this.title). کسی یک کاغذ به تو می‌دهد که رویش نوشته «کارگاه برنامه‌نویسی» (پارامتر title). تو کاغذ را می‌گذاری داخل جعبه — همین یعنی this.title = title.

چرا this لازم است؟

چون اسم پارامتر و اسم ویژگی یکی است؛ بدون this جاوا فکر می‌کند هر دو title به پارامتر اشاره می‌کنند و ویژگی خالی می‌ماند! this یعنی **«خودِ این شیء»**.

اگر اسم پارامتر مثلاً t بود (title = t;) دیگر this لازم نبود — ولی اسم‌های یکسان خوانایی بهتری دارند.

Event.java

متدهای Getter – خواندن اطلاعات

Getter یعنی گیرنده یا بخواننده؛ متدی ساده که فقط مقدار یک ویژگی را برمی‌گرداند – جواب می‌دهی، چیزی عوض نمی‌کنی.

Event.java

```
public String getTitle() {
    return title;
}

// همین الگو برای همه: getDate، getLocation، getCategory، getDescription
```

جزء	معنی
`public`	از بیرون قابل استفاده است
`String`	این متد یک متن برمی‌گرداند
`getTitle`	اسم متد (قرارداد: با `get` شروع شود)
`()`	هیچ پارامتری نمی‌گیرد
`return title;`	ویژگی `title` را برگردان

چرا مستقیم دسترسی نمی‌زنیم؟

چون ویژگی‌ها `private` هستند – `event.title` خطا می‌دهد (دسترسی مستقیم به خصوصی) ولی `event.getTitle()` کار می‌کند چون Getter عمومی است. Getter مثل دریچه‌ای است که فقط اجازه دیدن می‌دهد.

کجا استفاده می‌شود؟

```
txtTitle.setText(event.getTitle());
```

```
intent.putExtra(EXTRA_EVENT_TITLE, event.getTitle());
```

هر جا بخواهیم اطلاعات رویداد را بخوانیم، در `MainActivity` همین‌طور استفاده شده است.

Event.java

متدهای Setter – تغییر اطلاعات

Setter یعنی تنظیم‌کننده؛ متدی که مقدار یک ویژگی را تغییر می‌دهد – مثل این که کسی بگوید عنوان را عوض کن و تو کشو را باز کنی و محتوایش را عوض کنی.

Event.java

```java
public void setTitle(String title) {
    this.title = title;
}

// همین الگو برای همه: setDate، setLocation، setCategory، setDescription
```

جزء	معنی
`public`	از بیرون قابل استفاده است
`void`	چیزی برنمی‌گرداند (فقط کار انجام می‌دهد)
`setTitle`	اسم متد (قرارداد: با `set` شروع شود)
`(String title)`	یک پارامتر متن می‌گیرد
`this.title = title;`	مقدار پارامتر را داخل ویژگی بریز

در این جلسه هنوز استفاده نمی‌شوند!

در جلسات بعد برای ویرایش رویداد استفاده خواهند شد – یعنی الان برای آینده آماده‌اند. `this` در Setter دقیقاً مثل سازنده است.

تفاوت Getter و Setter در یک نگاه

وجه مقایسه	Getter	Setter
کار	می‌خواند	می‌نویسد (تغییر می‌دهد)
نوع بازگشتی	String (متن)	void (هیچی)
پارامتر	ندارد	دارد
اسم	با get شروع می‌شود	با set شروع می‌شود
تشبیه	«بپرس»	«دستور بده عوض کن»

چرا جدا گذاشتیم؟

۱ کنترل

اگر بعداً بخواهیم اعتبارسنجی اضافه کنیم (مثل «عنوان نباید خالی باشد»)، فقط Setter را تغییر می‌دهیم.

۲ انعطاف

می‌توانیم فقط Getter بدهیم و Setter ندهیم تا شیء تغییرناپذیر شود (در آینده).

۳ خوانایی

مشخص است چه چیزی قابل خواندن و چه چیزی قابل نوشتن است.

چرا این کلاس متد دیگری ندارد؟

`Event` یک کلاس مدل داده (Data Model) است؛ کارش فقط نگهداری اطلاعات است، نه نمایش و نه محاسبه.

اصل جداسازی مسئولیت‌ها — Separation of Concerns

Event

نگهداری اطلاعات (مدل)

MainActivity

نمایش و تعامل (View / Controller)

EventDetailActivity

نمایش جزئیات (View)

هر کلاس فقط یک کار انجام می‌دهد — این را «جداسازی مسئولیت‌ها» می‌گویند.

چرا خوب است؟

- هر کلاس ساده‌تر می‌شود
- هر کلاس تست‌پذیر می‌شود
- تغییر در یک کلاس کمتر روی بقیه اثر می‌گذارد

ارتباط Event با MainActivity — بخش اول

بخش ۱ از ۲

الف ساخت یک رویداد جدید

new Event(...)

MainActivity.java

```java
Event sampleEvent = new Event("کارگاه برنامه‌نویسی اندروید" ,"1405/08/25" ,"کارگاه کامپیوتر" ,"آموزشی" ,"آشنایی با ساخت اپلیکیشن‌ها");
```

- سازنده صدا زده می‌شود؛ یک شیء جدید ساخته می‌شود و اطلاعات داخلش ذخیره می‌شود.

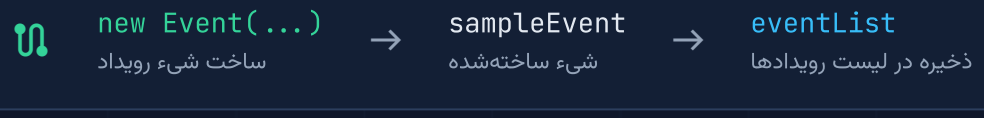
ب نگهداری در لیست

ArrayList<Event>

MainActivity.java

```java
private final ArrayList<Event> eventList = new ArrayList<>();
eventList.add(sampleEvent);
```

- ArrayList<Event> یعنی «لیستی از شیءهای Event» — هر عضو لیست یک رویداد کامل است.



ارتباط Event با MainActivity — بخش دوم

بخش ۲ از ۲

ج خواندن اطلاعات برای نمایش

getTitle()

MainActivity.java

```
txtTitle.setText(event.getTitle());
txtInfo.setText("تاریخ: " + event.getDate() + ...);
```

- اینجا Getterها صدا زده می‌شوند.

د ارسال به صفحه جزئیات

putExtra(...)

MainActivity.java

```
intent.putExtra(EXTRA_EVENT_TITLE, event.getTitle());
```

- باز هم Getter صدا زده می‌شود تا اطلاعات از شیء خوانده و به صفحه مقصد فرستاده شود.

نکته مهم — چرا خود شیء را نمی‌فرستیم؟

در openEventDetails اطلاعات تک‌تک با putExtra فرستاده می‌شوند، نه خود شیء Event؛ چون Intent فقط انواع ساده (متن، عدد و...) را می‌شناسد. برای فرستادن کل شیء باید کلاس Serializable یا Parcelable می‌شد — این را می‌توانیم به عنوان «برای جلسات بعد» بگوییم.

Serializable · Parcelable · برای جلسات بعد

جمع‌بندی سریع — کل کلاس در ۲ دقیقه

۶ نکته کلیدی

۱

این کلاس مدل یک رویداد است؛ هیچ چیزی نمایش نمی‌دهد، فقط اطلاعات را نگه می‌دارد.

۲

پنج ویژگی دارد: عنوان، تاریخ، محل، دسته‌بندی، توضیحات — همه `private` تا از دستکاری بیرونی محافظت شوند.

۳

سازنده با ۵ پارامتر، شیء را مقداردهی اولیه می‌کند.

۴

`Getter`ها برای خواندن و `Setter`ها برای تغییر اطلاعات استفاده می‌شوند.

۵

`this` در سازنده و `Setter` برای تفکیک ویژگی از پارامتر به کار می‌رود.

۶

یک کلاس مدل (`Model`) است و اصل جداسازی مسئولیت‌ها را رعایت می‌کند.

سوال‌های احتمالی کلاس — بخش ۱

بخش ۱ از ۲

س چرا ویژگی‌ها `private` هستند؟

ج برای محافظت و کپسوله‌سازی؛ از بیرون کسی نمی‌تواند مستقیم تغییرشان دهد و دسترسی باید از `Setter/Getter` عبور کند.

س چرا برای هر ویژگی هم `Getter` و هم `Setter` ساختیم؟

ج `Getter` برای خواندن است؛ `Setter` برای تغییر در آینده — الان استفاده نمی‌شود ولی برای ویرایش رویداد در جلسات بعد لازم است.

س `this` چیست؟

ج یعنی «خودِ این شیء»؛ وقتی اسم پارامتر و ویژگی یکی است، با `this` ویژگی را از پارامتر جدا می‌کنیم.

س تفاوت `public` و `private` چیست؟

ج `public` یعنی از همه‌جا قابل دسترسی؛ `private` یعنی فقط از داخل همان کلاس قابل دسترسی.

سوال‌های احتمالی کلاس – بخش ۲

س چرا `Event` وارث هیچ کلاسی نیست؟

ج چون یک کلاس مدل داده است و به امکانات خاصی نیاز ندارد؛ `MainActivity` وارث `AppCompatActivity` بود چون به امکانات `Activity` نیاز داشت.

س چرا `String` استفاده کردیم و نه نوع داده دیگری؟

ج چون همه اطلاعات (عنوان، تاریخ، محل، دسته، توضیحات) متن هستند؛ تاریخ هم فعلاً متن گرفته شده و بعداً می‌توان به نوع دقیق‌تری تبدیلش کرد.

س اگر ویژگی‌ای را فراموش کنیم مقداردهی کنیم چه می‌شود؟

ج سازنده هر ۵ پارامتر را اجباری می‌کند؛ نمی‌توان بدون دادن همه مقادیر شیء ساخت – یک محدودیت مفید است.

س تفاوت سازنده و متد معمولی چیست؟

ج سازنده اسمش با کلاس یکی است و نوع بازگشتی ندارد و با `new` خودش اجرا می‌شود؛ متد معمولی اسم دلخواه دارد و باید دستی صدا زده شود.



«Event قلب اطلاعات برنامه است؛ MainActivity دست و چشم برنامه است.
قلب بدون دست کار نمی‌کند، دست هم بدون قلب اطلاعاتی ندارد.»

متشکرم – آماده‌ی پاسخ به سوال‌های شما هستم.

com.mehrsamweb.universityevents · Event.java